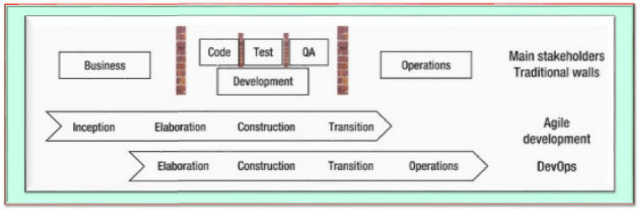
# Agile Vs DevOps

innovate achieve lead

## Traditional, Agile and DevOps – A comparison

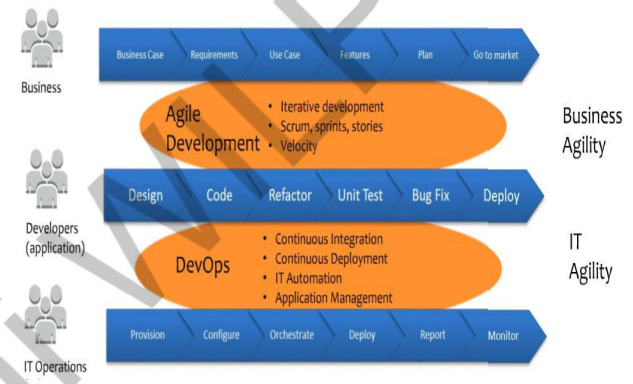



- Agile breaks the wall between Business and Development team.
- DevOps breaks the wall between Development and Operations team.
- DevOps centers on the concept of sharing: sharing ideas, issues, processes, tools and goals.



# Agile Vs DevOps

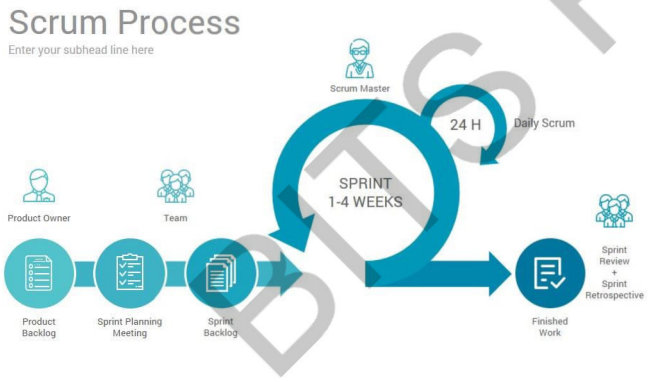
innovate achieve lead






# Scrum

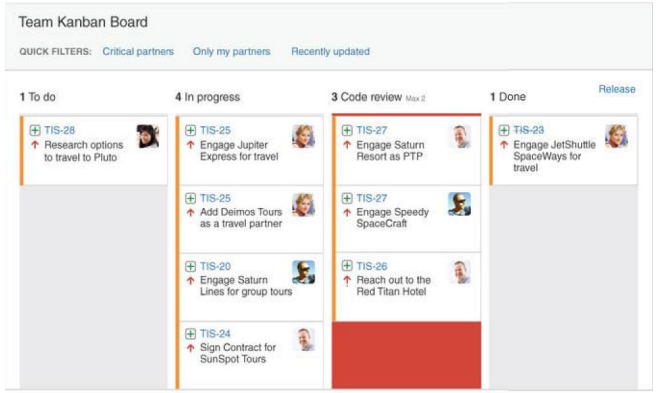
innovate achieve lead






# Test Driven Development

innovate achieve lead



## Scrum vs Kanban (Non flow vs flow based)

| | SCRUM | KANBAN |
|---|---|---|
| Cadence | Regular fixed length sprints (ie, 2 weeks) | Continuous flow |
| Release methodology | At the end of each sprint if approved by the product owner | Continuous delivery or at the team's discretion |
| Roles | Product owner, scrum master, development team | No existing roles. Some teams enlist the help of an agile coach. |
| Key metrics | Velocity | Cycle time |
| Change philosophy | Teams should strive to not make changes to the sprint forecast during the sprint. Doing so compromises learnings around estimation. | Change can happen at any time |

**Velocity** measures completed stories per iteration. The unit is work per **time**, e.g. 4 stories per 2 week iteration. **Cycle time** measures the amount of **time** passed working on a story.



## Iteration- and Flow-Based Agile

### Iteration based Agile

- ☐ Requires an initial prioritization of features, which then allows the team to start work by progressing from the highest priority feature down to the lowest.
- ☐ The iterations are regularly spaced out in constant, equal intervals (thus the name) which then builds up a cycle that the team becomes familiar with.
- ☐ A single iteration may have multiple features, but features will be carried over to the next iteration if it is incomplete.

### Flow based Agile

- ☐ Team assesses the features backlog and then decide on which feature to work on based on their available capacity.
- ☐ Since there is no fixed time interval for releases, the team togetherwith the stakeholders will need to decide on a schedule for planning, review and testing.



## Enterprise Agile Frameworks

- Aims to scale Agile principles and practices to the enterprise, and address the specific challenges of managing a large number of Agile large-size teams
- How to apply methodologies such as Scrum or Kanban which are designed for small teams, to organizations that count hundreds of teams, teams-of-teams, or even teams-of-teams-of-teams

- SAFe - Scaled Agile Framework
- DAD - Disciplined Agile Delivery
- Nexus
- LeSS Huge - Large Scale Scrum



## TDD, FDD, BDD

### TDD (Test Driven Development)

- Writing a test first, watching it fail then writing enough of the implementation to make it pass then re-factor originally seemed like an alien concept that was going to make the development process longer
- How functionality implemented?

### FDD (Feature Driven Development)

- splitting up a project into features but involves prioritization and planning
- Think about and begin modelling a system before any development begins

### BDD (Behavior Driven Development)

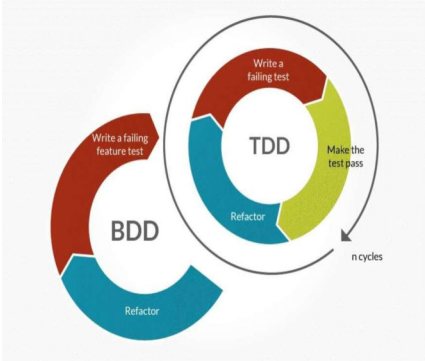
- BDD is an extension upon TDD and does not contest the fundamental values of TDD.
- Process starts by writing a scenario as per the expected behavior.
- BDD, a test is written that can satisfy both the developer and customer.
- How application implemented?





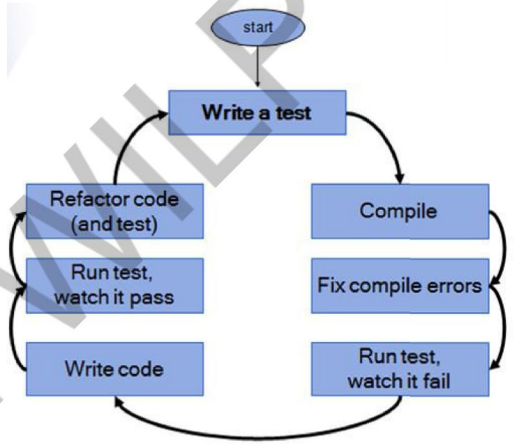
innovate achieve lead

## TDD, FDD, BDD



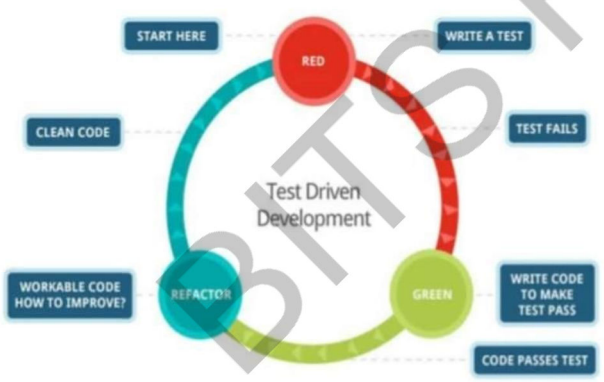



## TDD- Test Driven Development






## TDD- Test Driven Development






## FDD - Feature Driven Development

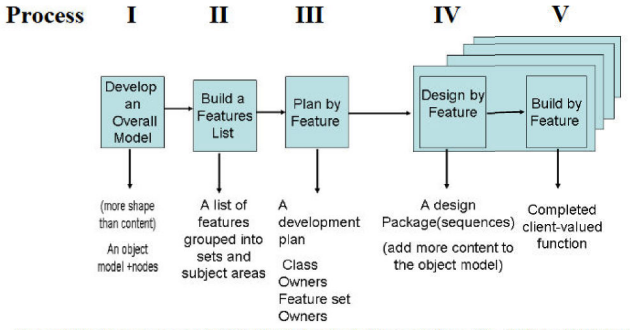
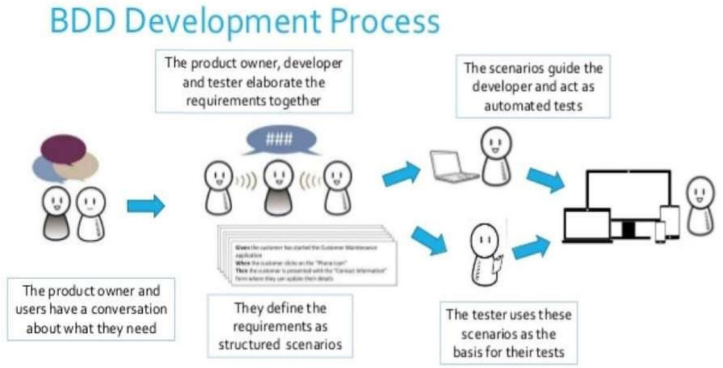



**Figure 3 The five processes of FDD with their outputs** (Source: Palmer, SR., Felsing , JM.2002,p.57).

## BDD – Behaviour Driven Development






## BDD – Behaviour Driven Development

### Behavior Driven Development/Testing

- What is BDD?
  - The main focus is on the expected behavior of the application and it's components.
  - User stories created and maintained collaboratively by all stakeholders
- What are the benefits?
  - Define verifiable, executable and unambiguous requirements
  - Developing features that truly add business value
  - Preventing defects rather than finding defects
  - Bring QA involvement to the forefront, great for team dynamics



## BDD – Behaviour Driven Development

**Here is a simple example**

**Feature:** log into the system. User should be signed into the system when he provides valid username and password
**Scenario:** Successful login with Valid Credentials

**Given** User is at the Home Page
**And** Navigate to LogIn Page
**When** User enters credentials
| username | password |
| testuser_1 | Test@123 |
**And** User login into a system
**Then** user is on the main profile page



## DevOps– Practices

- Key DevOps Practices

- Release planning
- Continuous integration
- Continuous delivery
- Continuous testing
- Continuous monitoring and feedback

# DevOps– Practices

innovate achieve lead

## 1. Release planning

- Release planning is a critical business function, driven by business needs to offer capabilities to customers and the timelines of these needs.
- Businesses require well defined release planning and management processes that drive release roadmaps, project plans, and delivery schedules, as well as end-to-end traceability across these processes.
- Traditionally, release planning accomplished by using spreadsheets and holding meetings (often, long ones) with all stakeholders across the business to track all business needs applications under development, their development status, and release plans.
- Well-defined processes and automation, however, eliminate the need for those spreadsheets and meetings, and enable streamlined and enable predictable releases.
- Leveraging lean and agile practices also results in smaller, more frequent releases, permitting enhanced focus on quality.



# DevOps– Practices

innovate achieve lead

## 2. Continuous integration

- ☐ Allowing large teams of developers, working on cross-technology components in multiple locations, to deliver software in an agile manner.
- ☐ Each team's work is continuously integrated with that of other development teams and then validated.
- ☐ Continuous integration reduces risk and identifies issues earlier in the software development life cycle.

## 3. Continuous Delivery/Deployment

- ☐ Continuous integration naturally leads to the practice of continuous delivery/deployment
- ☐ Process of automating the deployment of the software to the testing, system testing, staging, and production environments.
- ☐ Automated process in all environments to improve efficiency and reduce the risk introduced by inconsistent processes



# DevOps– Practices

innovate achieve lead

## 4. Continuous Delivery/Deployment

- Processes that they adopt may vary from project to project, based on individual testing needs and on the requirements of service level agreements.
- Adopt processes in three areas:

(1) Test environment provisioning and configuration
(2) Test data management
(3) Integration, functional, performance, and security testing



# DevOps– Practices

innovate achieve lead

## 5. Continuous monitoring and feedback

- Customer feedback comes in different forms, such as tickets opened by customers, formal change requests, informal complaints, and ratings in app stores.
- Feedback also comes from monitoring data. This data comes from the servers running the application; from Development, QA, and Production; or from metrics tools embedded in the application that capture user actions.
- Businesses need well-defined processes to absorb the feedback from myriad sources and incorporate them into software delivery plans.

# DevOps– Practices

innovate achieve lead

### 6. Continuous Improvement

- Process adoption isn't a one-time action, but an ongoing process.
- An organization should have built-in processes that identify areas for improvement as the organization matures and learns from the processes it has adopted.
- Many businesses have process improvement teams that work on improving processes based on observations and lessons learned.
- Teams that adopt the processes to self-assess and determine their own process improvement paths.



# DevOps– Tools

innovate achieve lead

- Enables people to focus on high-value creative work while delegating routine tasks to automation. Technology also allows teams of practitioners to leverage and scale their time and abilities.
- Organization has to reuse assets, code, and practices to be cost-effective and efficient.
- Standardizing automation also makes people more effective
- common set of tools allows practitioners to work anywhere, and new team members need to learn only one set of tools and a process that's efficient, cost-effective, repeatable, and scalable.
- Categorized into major areas are
  - Infrastructure as Code
  - Delivery Pipeline
  - Deployment automation and release management



# DevOps– Cloud as a catalyst

innovate achieve lead

- Cloud is key enabler and has affected DevOps. It helps DevOps to mature.
- In traditional organizations that use physical servers, requesting a new server is a complex, time-consuming process. It may take days or even weeks to requisition a new physical server, install and set up the operating system (OS), and configure all the software required to deploy the application.
- Pervasiveness of cloud technology, private, public, and hybrid, has enabled organizations to provision environments on demand.
- Virtualization in Cloud solve that problem to a large extent. This has led to tremendous growth in the scale by which the environments to which applications are deployed are used.
- This scale has necessitated automation on software deployment tasks, which DevOps addresses
- Ability to create and switch environments simply, the ability to create VMs easily, and the management of databases etc..



# DevOps– Cloud as a catalyst

innovate achieve lead

- Advent of cloud actually facilitates continuous deployment and provides easier access to production like environments during the development and testing stages of the delivery pipeline.
- Developer can not only invoke an automated build, but also request the provisioning and configuration of a production like environment in the cloud and then deploy the application being built to it without any direct involvement by the operations team.

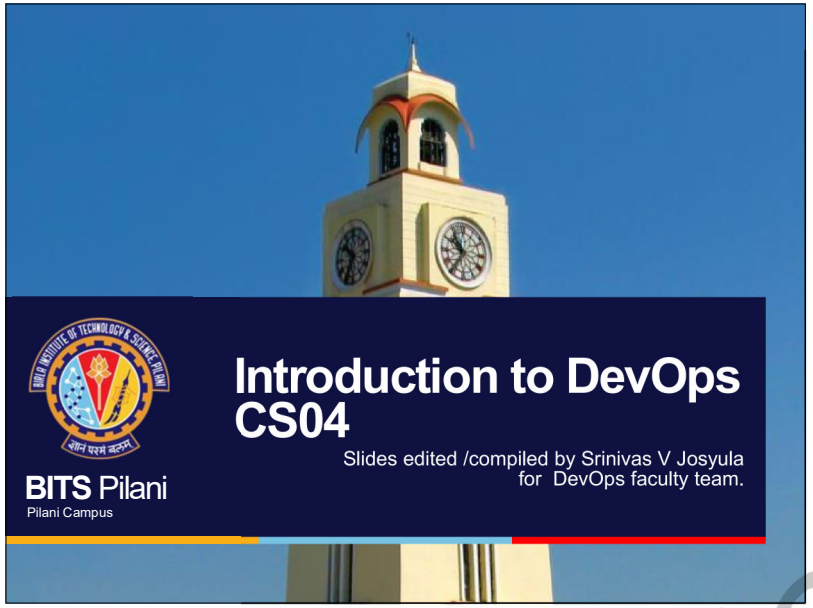
innovate achieve lead

## IMP Note to Self


innovate achieve lead

## Thank you

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES




## IMP Note to Self

innovate achieve lead





## CSIW ZG515, Introduction to DevOps
## CS 04: DevOps Dimensions?



## IMP Note to Students

innovate achieve lead

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**





innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Coverage

- DevOps – People
  - Building competencies, Full Stack Developers
- Self-organized teams, Intrinsic Motivation
  - Technology in DevOps(Infrastructure as code, Release tools, Delivery pipeline, technology as enablers for devOps)
- Delivery Pipeline, Release Management)
  - Tools/technology as enablers for DevOps
  - Discuss on Cloud as a catalyst for DevOps



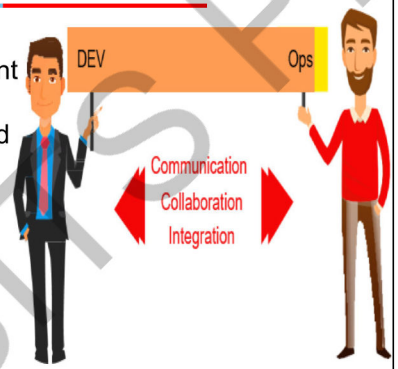
## Recorded Lectures for CS05

| |
|---|
| RL2.3 Devops – People |
| RL2.3.1 Team structure in a DevOps |
| RL2.3.2 Transformation to Enterprise DevOps culture |
| RL2.4 Devops Tools |
| RL2.4.1 Tools and Technology in DevOps |
| RL2.4.2 Cloud as a catalyst for DevOps |



## What is DevOps?

DevOps is a software development method that stresses communication, collaboration and integration between software developers and information technology (IT) operation professionals.




## Roles In DevOps

Team Lead / Project lead / Scrum Master
Team Member
Scrum Master
Additional Roles
- Service owner
- Reliability engineer
- Gatekeeper
- And DevOps engineer



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Focuses on

Small teams
- The size of the team should be relatively small
- Amazon has a “two pizza rule” (2 pizzas should suffice the team)

The advantages of small teams are
- Quick decision making, allows speaking up
- Team members who are to be motivated, take ownership

The disadvantage
- Large tasks – May have to be accomplished by a small number of resources
- Task has to be broken up into smaller pieces
- A small team, by necessity, works on a small amount of code

- Individuals and Interactions as opposed to Processes & tools
- Working software as opposed to Documentation
- Customer collaboration as opposed to Contracts
- Responding to change as opposed to Extensive planning

## Component vs Feature teams

- Component
  - Time elapsed
  - Expertize in UI, M/W and DB – Each acting as a silo
- Feature teams
  - Multi faceted individuals who can work in any area of system
  - At least 1 team member who has specialist knowledge for each layer of system
  - Allow teams to work on vertical slices of system

## Food for thought

- Have your developers spend the weekend with operations, watching a production rollout and learning what they go through.
- Likewise, track how many steps or service tickets it takes for a developer to request a new virtual system.

## 7Cs of DevOps

- Communication
- Collaboration
- Controlled Process
- Continuous Integration
- Continuous Deployment
- Continuous Testing
- Continuous Monitoring



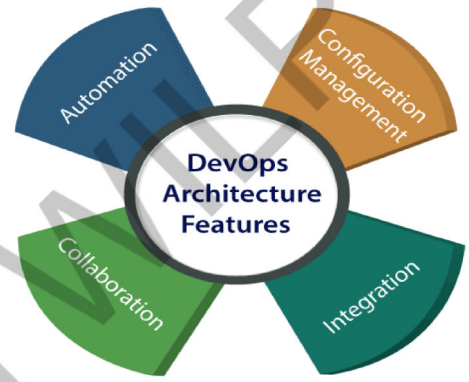


innovate achieve lead

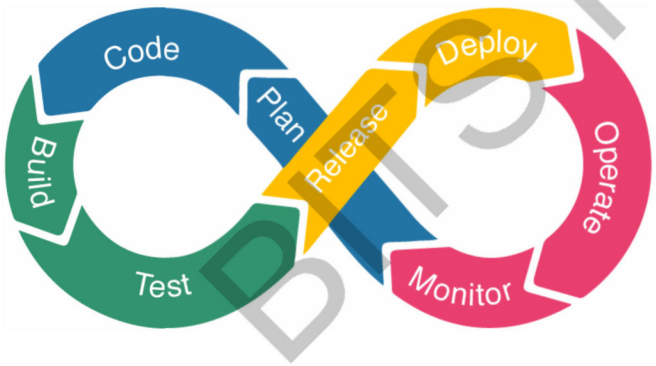
## People Dimension

- Regular 1 o 1
- Feedback
- Blameless culture!!
- Can you measure org culture?
- Tools
  - Local Dev Environment
  - Version Control
  - Artifact management
  - Automation tools – Server installation, Configuration management
  - System provisioning
  - Test & build automation

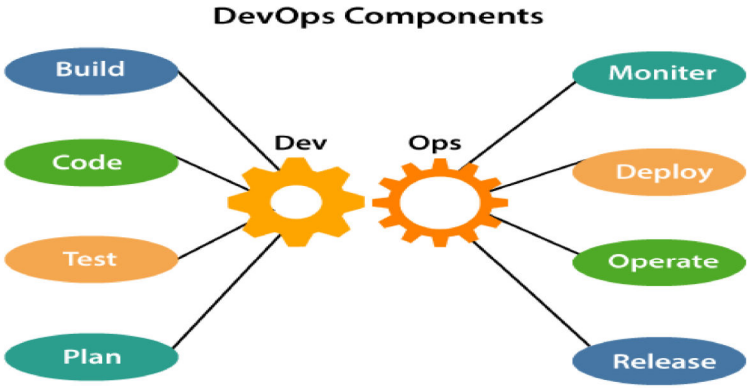
## DevOps Features




## DevOps Phases




## DevOps Lifecycle Components



## DevOps Components

Plan
- DevOps use Agile methodology to plan the development With the operations and development team in sync, it helps in organizing the work to plan accordingly to increase productivity

Build
- DevOps, the usage of cloud, sharing of resources comes into the picture, and the build is dependent upon the user's need, which is a mechanism to control the usage of resources or capacity.

Code
- Many good practices such as Git enables the code to be used, which ensures writing the code for business, helps to track changes, getting notified about the reason behind the difference in the actual and the expected output, and if necessary reverting to the original code developed. The code can be appropriately arranged in **files, folders**, etc. And they can be reused

## DevOps Components

- Test
  - The testing can be automated, which decreases the time for testing so that the time to deploy the code to production can be reduced as automating the running of the scripts will remove many manual steps.
- Monitor
  - Continuous monitoring is used to identify any risk of failure. Also, it helps in tracking the system accurately so that the health of the application can be checked. The monitoring becomes more comfortable with services where the log data may get monitored through many third-party tools such as Splunk.



## DevOps Components

Deploy

Many systems can support the scheduler for automated deployment. The cloud management platform enables users to capture accurate insights and view the optimization scenario, analytics on trends by the deployment of dashboards

Operate

DevOps changes the way traditional approach of developing and testing separately. The teams operate in a collaborative way where both the teams actively participate throughout the service lifecycle. The operation team interacts with developers, and they come up with a monitoring plan which serves the IT and business requirements



## Why do you want to transform?

- The intent is to transform the way solutions are being delivered today and to do it faster, cheaper and better
- The techniques of Agile and DevOps have been successfully implemented in many large and small organizations to enable those outcomes





innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Devops – As an Agile enabler

innovate achieve lead

Gartner defines: A business driven approach to deliver customer value by using Agile Methods, collaboration and automation

Eco system of Agile methodology, DevOps, increased focus on collaboration = Customer Experience



## Transforming the Enterprise - DevOps

innovate achieve lead

1. Current state of DevOps maturity
   1. Assess
   2. Define the Desired State
2. Develop a transformation plan
3. Implement the transformation plan
4. Measure and monitor progress

**Key challenges**

1. Cultural change
2. Process change Technology change:
3. investment in new tools and technologies.



## Infrastructure as Code

innovate achieve lead

- IT Departments have been busy providing IT solutions for their customers, but they forgot their own backyard
- IT departments themselves became bottlenecks
- No one was trying to automate the IT Environment itself



## Infrastructure as Code

innovate achieve lead

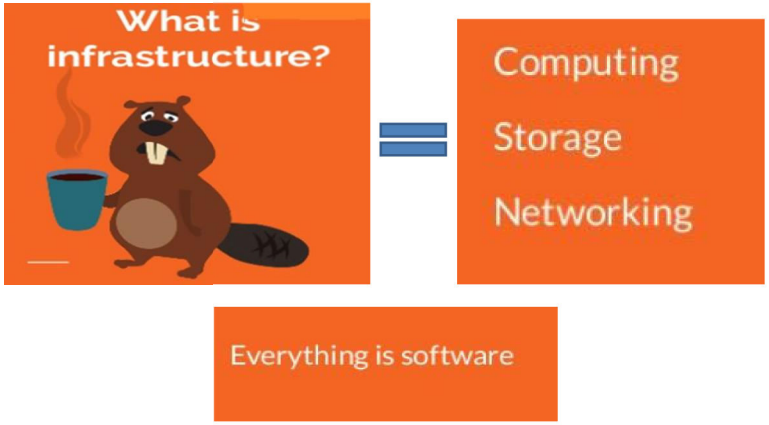
"Infrastructure as code represents the idea that everything that needs to run as infrastructure can be considered as software"





innovate achieve lead

## Infrastructure as Code






## Infrastructure as Code

- Infrastructure as code (IaC) is an IT practice that uses programming languages and machine-readable code to manage and configure computing infrastructure
- Minimize manual configuration
- IaC helps developers and operations teams automatically manage, monitor, and provision resources, and improve infrastructure consistency, security, automation, and performance



## What is the impact?

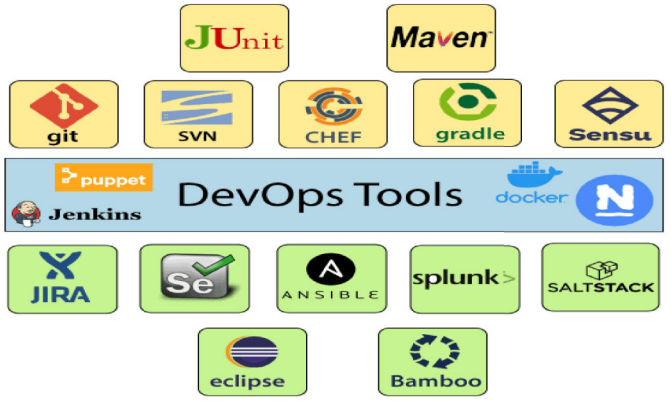
### What is the impact ?

- Customers who embraced this new way of building infrastructure for servers observed:

| 200x | 24x | 3x | 2.5x |
|---|---|---|---|
| more frequent deployment | faster recovery from failure | lower change failure rate | Shorter lead time |



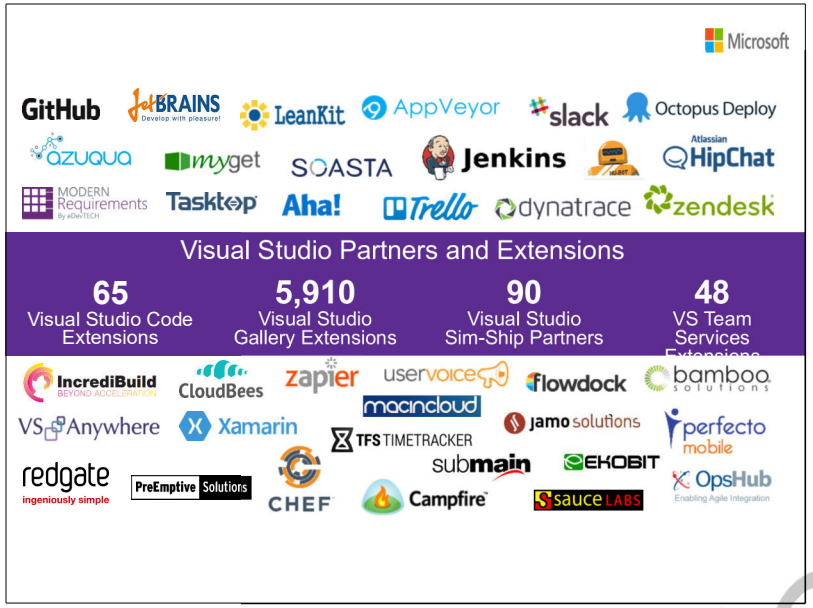
## DevOps stack








innovate achieve lead




## Deployment Pipeline

- The purpose of a pipeline is to transport some resource from point A to point B effectively with minimum upkeep or attention required once built
- *The purpose of a build pipeline is to transport the code from a developer machine or environment to an environment quickly and effectively with minimum upkeep or attention required once built*



## Deployment Pipeline

The deployment pipeline

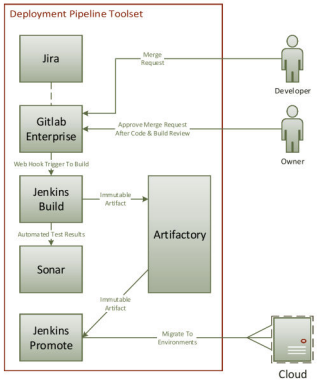



## Deployment Pipeline

- Build pipelines require measurements and verification to ensure
  - The Product is high quality
  - Meets the Standards
- Promotion pipeline
  - The promotion pipeline promotes inventory entries from one environment to another and creates a promotion pull/merge request.

## Deployment Pipeline benefits

- Improved Developer Experience
- Delivering Software At An Agile Pace
- Continuous/Fast Feedback
- Approvals To Audit
- Secure At The Start
- Automated testing
- Industry Standard Best Practices
- Improved time to value through code enabled deployments
- Reduced support footprint due to consistent deployments
- Simple automated deployments to any target zone
- Automation, Automation, Automation






## Cloud is no longer a mystery

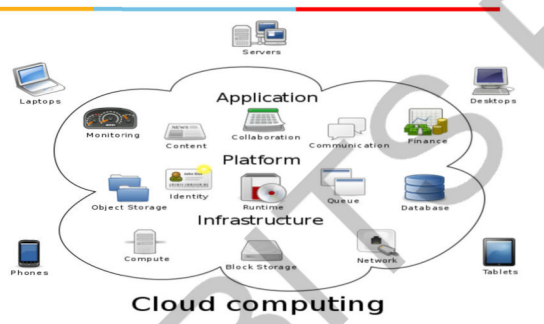
- Not quite!!
  - Not the concept of the cloud but why they are not seeing the benefits that were promised to them
  - The focus is on adopting cloud-based infrastructure and managing it
  - While we will discuss cost benefits primarily, one should not forget the other benefits and risks of the cloud

Benefits:
- Redundancy for resilience
- The scalability of resources
- The strong ecosystem of related

Risks:
- The dependency on a third-party provider
- The challenges with data sovereignty
- Risk of being attacked because you are on a popular platform



## Cloud is no longer a mystery

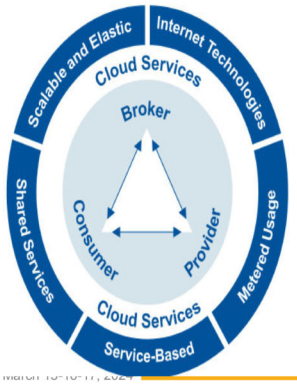


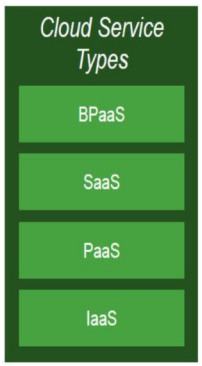

***Cloud computing metaphor: the group of networked elements providing services need not be individually addressed or managed by users; instead, the entire provider-managed suite of hardware and software can be thought of as an amorphous cloud.***

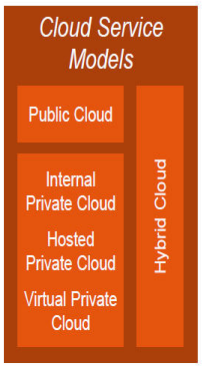


## Cloud – Providing IT capabilities as a service

*A style of computing where scalable and elastic IT-related capabilities are provided "as a service" to customers using Internet technologies.*



## IMP Note to Self




## The End

Thank you

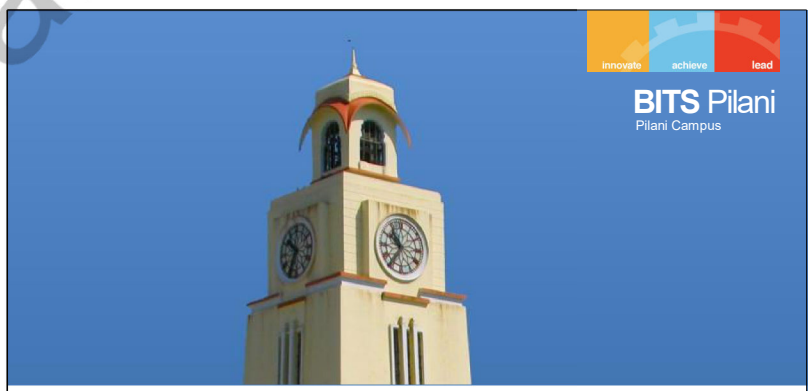
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES


## IMP Note to Self




## IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**




# CSIW ZG515, Introduction to DevOps
# CS 05 :Version Control Systems

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Coverage

- Centralized Version Control Systems
- Distributed Version Control Systems
- Overview of GIT
- Git Feature branching
- Managing Conflicts using GIT
- Tagging and Merging operations in GIT
- Benefits of Clean code

References

T1 - Chapter 3,

R1 - Chapter 5



## Centralized Version Control Systems

- A centralized version control system offers software development teams a way to collaborate using a central server.
- A server acts as the main centralized repository which stores every version of code.
- Using centralized source control, every user commits directly to the main branch, so this type of version control often works well for small teams, because team members have the ability to communicate quickly so that no two developers want to work on the same piece of code simultaneously
- As a client-server model, a centralized workflow enables file locking so that any piece of the code that's currently checked out will not be accessible to others, ensuring that only one developer can contribute to the code at a time. Team members use branches to contribute to the central repository, and the server will unlock files after merges.



## Distributed Version Control Systems

- A distributed version control system (DVCS) brings a local copy of the complete repository to every team member's computer, so they can commit, branch, and merge locally. The server doesn't have to store a physical file for each branch — it just needs the differences between each commit.
- Distributed source code management systems, such as Git, Mercurial, and Bazaar, mirror the repository and its entire history as a local copy on individual hard drives.
- Distributed version control systems help software development teams create strong workflows and hierarchies, with each developer pushing code changes to their own repository and maintainers setting a code review process to ensure only quality code merges into the main repository.
- A contributor can no longer rely on a server to resolve conflicts when merging and has to resolve them locally, which can result in confusing merge commits. However, despite the initial discomfort, a distributed source control system can ensure stable code development when multiple developers contribute to software development projects.



## Introducing GitHub

- A web-based hosting and collaboration platform used by professional programmers to develop new software.
- Professional who uses GitHub can create and manage software with the advanced platform.
- The largest and most advanced global collaboration platform where developers and companies build and maintain their software using the Git version control tool.
- Used by software engineers, programmers, developers, instructors, and coding students to build unique code.





innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Continue-GitHub Intro

- The biggest source code host with over 200 million repositories.
- Storage space where your projects are stored.
- Companies and professionals host, store and record their code in the Github cloud.
- Hosting is free with an unlimited private and public remote repository.
- Users can take advantage of the benefits of version history with revision control. This feature, along with pull requests, make collaboration on code files transparent and easier to trace.



## Why Gut Hub Is popular

- Github started as a small startup company in 2008 and made it to **Forbe's Unicorn List of companies**, valued at more than $1 billion in a little over ten years.
- Some facts that support Github's surge in popularity over the years.
  - **Second leading traffic referrers to LinkedIn worldwide 2021**. As of June 2021, Github generated more than 5.09% of the referral traffic to LinkedIn.
  - **84% of Fortune 100 companies use Github Enterprise**. Many big companies have seen the advantages of using Github Enterprise to help streamline their security, administration, and collaboration efforts.



## Why Gut Hub Is popular

- **73 million software developers are using Github**. The company had 56 million users in December 2020, but the community just keeps growing. The running count as of 2021 is 73 million users. Github projects an increase to 100 million developers in the next five years.



## Purpose of Github

- Github is used by developers, programming instructors, students, and enterprises worldwide to create millions of open source projects and enable organized collaboration in one platform.
- It is a collaborative web-based platform with version control systems that offers a more efficient way to build excellent software





innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## GitHub Features

**Stores Your Code in the Cloud**

- gives your code a home in the cloud
- can record all coding changes in the platform, making it easier to review, trace, and retract changes if necessary.
- Hosting code is free, and you can start creating repositories for free, even with a basic personal account

**Collaboration Tool**

- makes project management easier
- Let's your colleagues work individually on separate branches on the platform.



## GitHub Features

- allows you to merge the code in a definitive branch called the mast.er branch



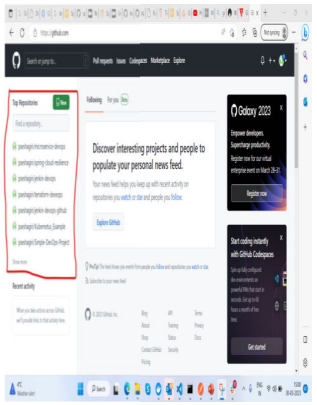
## Components

- Repositories
- Branches
- Commits
- Pull Requests
- Git (the version control tool GitHub is built on)



## Repository

- **a repository stores everything pertinent to a specific project including files, images, spreadsheets, data sets, and videos, often sorted into files.**
- **. git/ folder inside a project**. This repository tracks all changes made to files in your project, building a history over time.
- **each file's revision history**.
- manage your project's work within the repository.
- **lets you and others work together on projects from anywhere**.

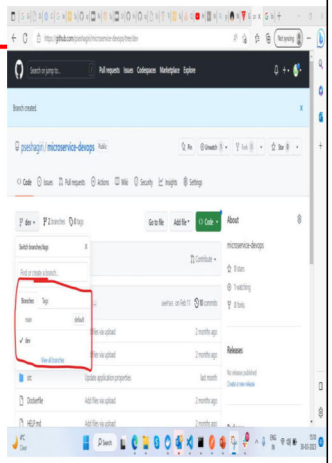







innovate achieve lead

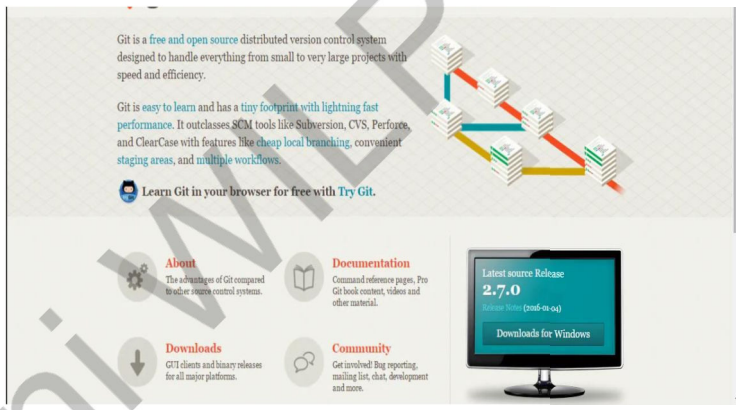
## Branches



- ✓ way to work on a new feature without affecting the main codebase.
- ✓ **is a feature of version control systems**, which means that Git tracks at which point in the version history you created it.
- ✓ Git lets you merge a branch back into mainline, merging them.

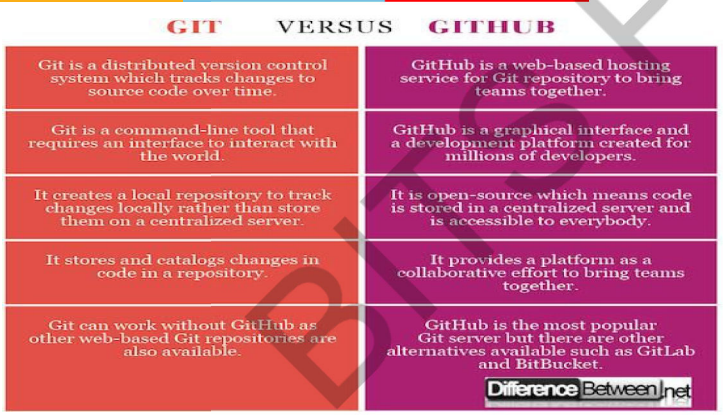


## Git






## Git vs GitHub



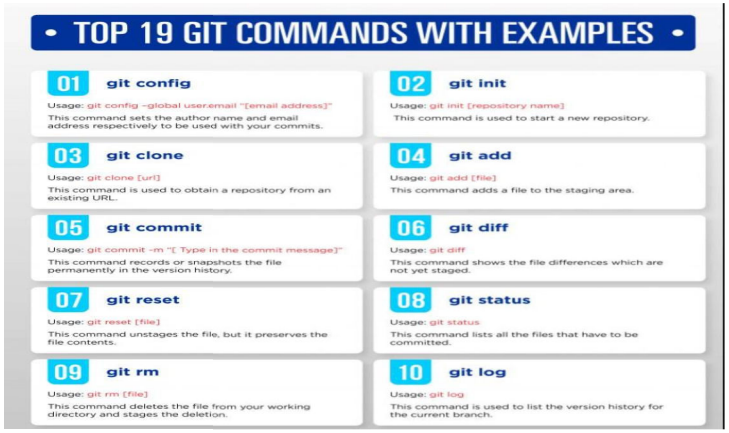



## Git Commands

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Git Commands

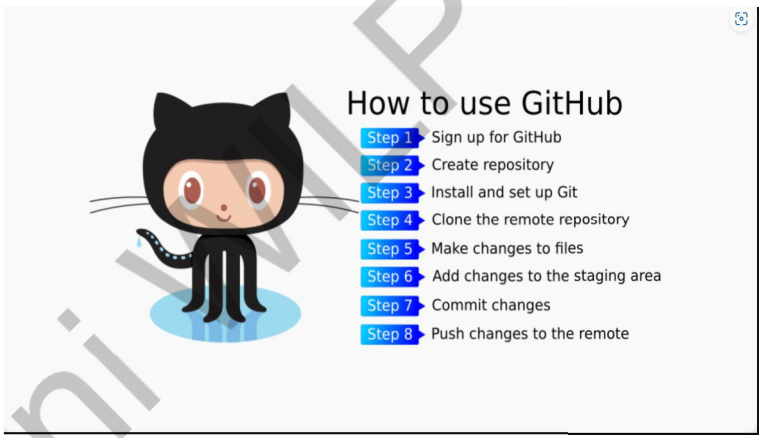
innovate achieve lead






## LAB SESSION

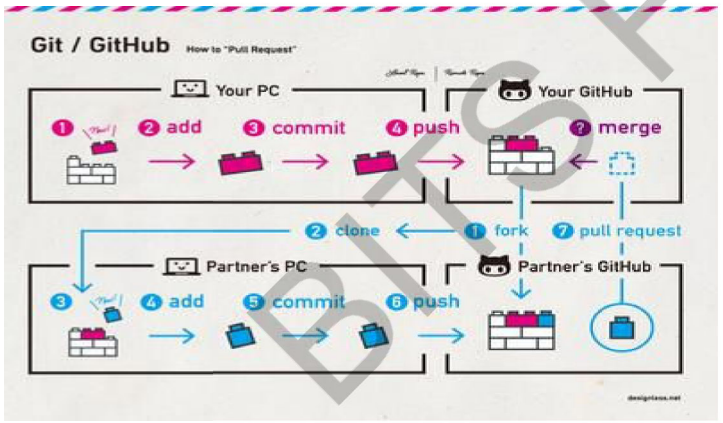
innovate achieve lead






## Commands

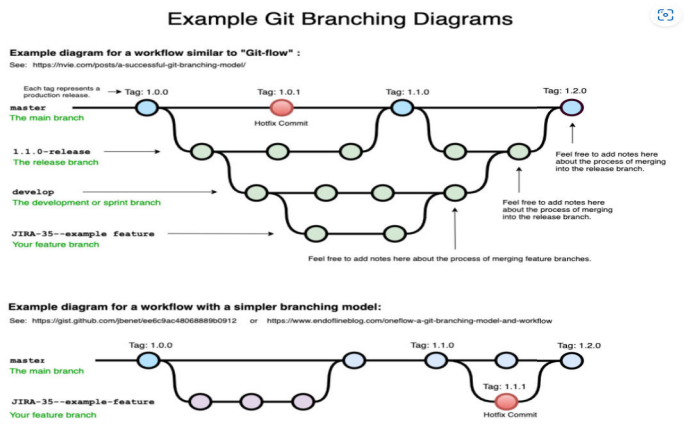
innovate achieve lead






## Branching Strategy

innovate achieve lead

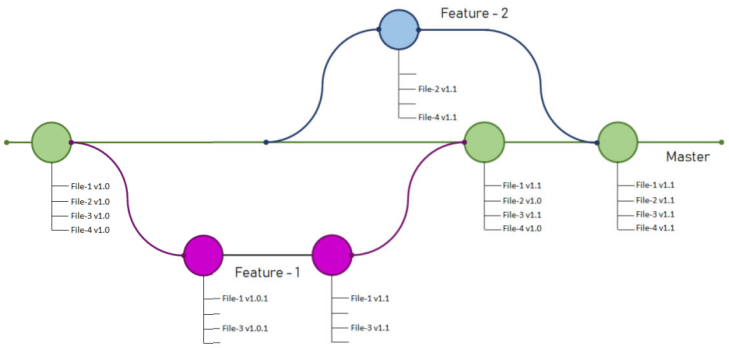







innovate achieve lead

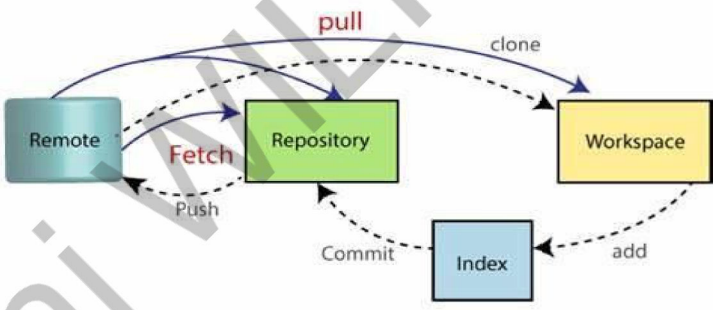
## Branching Example-2






## Pull Example

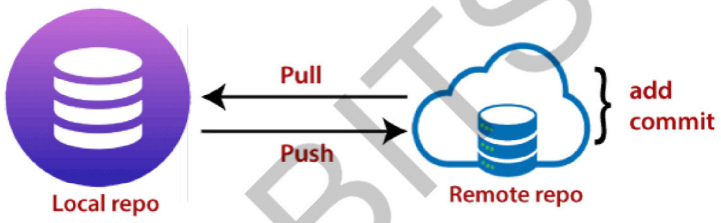
Git Pull





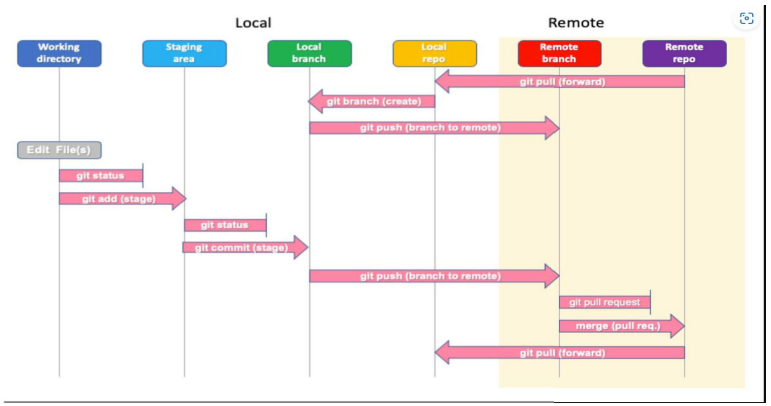

## Git Push Example

Git Push






## OverAll Workflow



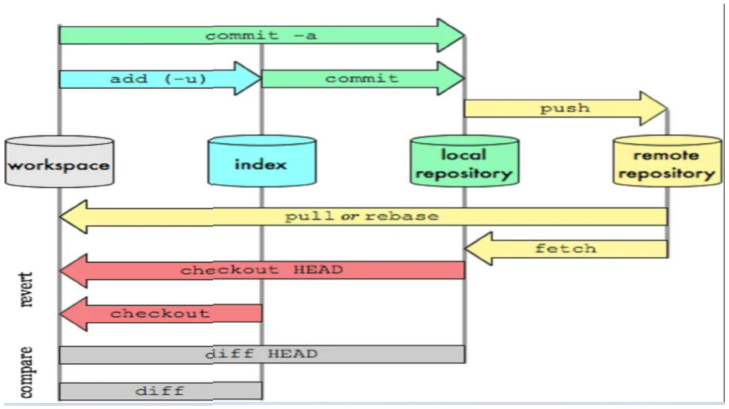





innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Git Data Transport Command






## Cooperate usage

| Companies That Use Github | Who Uses Github at This Company? | What Does This Company Use Github For? | Estimated Number of Employees |
|---|---|---|---|
| 3M Corporation | Information analyst, software engineer | Centralize code and teams to speed up research and development of new digital technologies | 95,000 |
| Adobe Systems Inc. | Software engineer, computer scientist, software development engineer | To make legal and technical processes more efficient | 22,516 |
| BuzzFeed | IT engineer, IT manager, principal engineer | For software engineers and news team to collaborate in one platform | 1,840 |
| Coinbase Global, Inc. | Software engineer, risk manager | For organization-wide transparency | 1,249 |
| Dell Technologies | Technical support engineer, Java developer, principal software engineer | To work on bigger projects and finish them faster | 158,000 |



## Cooperate usage

| | | | |
|---|---|---|---|
| Facebook | Research scientist, software engineer, computer vision engineer | To give back to the developer community. Their Github projects React and PyTorch are some of the most-followed projects in the Github platform. | 58,604 |
| Ford Motor Company | Software engineer, product manager, data scientist | Collaboration work on complex car features | 186,000 |
| LinkedIn | Instructional designer, senior software engineer, front end software engineer | To manage end-to-end workflow | 18,000 |
| Procter & Gamble | IT manager, senior software engineer | To update DevOps and CI/CD tooling and processes | 101,000 |
| Seagate | Senior staff engineer, staff engineer | For its open-source software, CORTX | 40,000 |



## IMP Note to Self





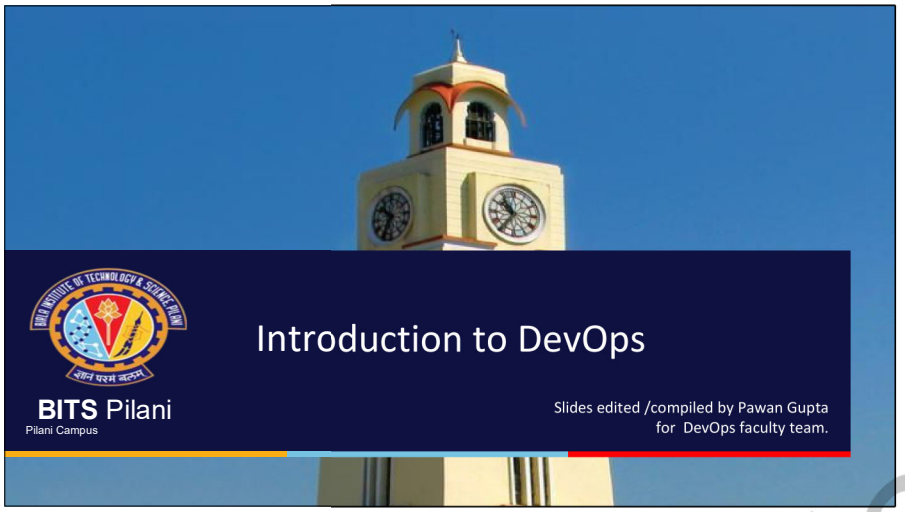

innovate achieve lead

## The End

innovate achieve lead

Thank you

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES




## IMP Note to Self


## IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**


# CSIW ZG515, Introduction to DevOps

## CS 06 : DevOps – Continuous Build and Code Quality



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Coverage

| | |
|---|---|
| · **Continuous build and code quality**<br>· Manage Dependencies<br>· Automate the process of assembling software components with build tools<br>· Use of Build Tools- Maven, Gradle<br>· Unit testing<br>· Enable Fast Reliable Automated Testing<br>· Setting up Automated Test Suite – Selenium<br>· Continuous code inspection - Code quality<br>· Code quality analysis tools- sonarqube | T1 - Chapter 5,<br>T2- Chapter 4, 6, 13<br>R2-Chapter 3 |



## Build Management

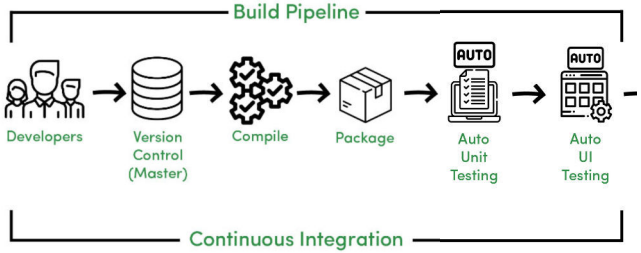
## Introduction to build

In software development, building refers to the process of converting the source code of a software application into an executable form that can be run on a computer. Building is an important step in the software development lifecycle, and it involves compiling, linking, and packaging the software.

However, the general steps involved in building a software application include:

1. **Compiling**: This involves translating the source code of the software into machine code that the computer can understand. This step is typically done using a compiler, which can generate object files from the source code.
2. **Linking**: This involves combining the object files generated during the compilation step into a single executable file. The linker resolves external references between object files and adds any necessary libraries to the executable.
3. **Packaging**: This involves creating a package or distribution of the executable and any necessary resources, such as configuration files, documentation, and other dependencies.

## Continuous Integration – Build Process



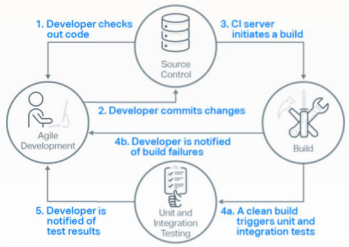



innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Continuous Integration Build Workflow

The following diagram summarizes the key steps in a continuous integration process.




## Build Process Steps

1. Developers check out code into their private workspaces. They then make their changes and test them locally.
2. When done, developers check in their changes into the source control repository.
3. The CI server monitors the source control repository and when it detects a change it triggers a build of the relevant sources.
4. After a successful build, the CI server performs some or all of the following activities:
    1. makes deployable artefacts available for testing
    2. assigns a build label to the version of the code that was just built
    3. notifies the relevant team members that a successful build occurred
    4. triggers unit and integration testing
5. At this point, the changes that were checked in at step 2 have been successfully built and a build label has been applied to the source code that was used for the build, meaning that the build could be recreated if necessary.
6. In the event of a build failure, the CI server sends notifications to the relevant developers who restart the process from step 1 to make the changes necessary to resolve the build errors.
7. After the unit and integration testing has taken place, the relevant team members are notified of the test results.

## Build Tools

There are many tools available for automating software builds. Here are some popular ones:

**Jenkins**: Jenkins is a widely used open-source automation server that helps automate building, testing, and deploying software projects. It supports multiple programming languages and has a vast collection of plugins that can be used to extend its functionality.
https://www.jenkins.io/

**Gradle:** Gradle is a popular build automation tool that is designed to be flexible and efficient. It supports multiple programming languages and platforms, including Java, Groovy, and Kotlin. It has a plugin-based architecture that allows you to customize and extend its functionality.
https://gradle.org/

**Maven:** Maven is another popular build automation tool that is widely used in the Java community. It provides a simple and consistent way to build and manage projects, including support for dependency management, project reporting, and more.
https://maven.apache.org/

## Gradle Build Cycle




innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

## Build Process Steps using Gradle

The Gradle build process typically involves the following steps:

1. **Initialization**: Gradle initializes the project by reading the settings.gradle and build.gradle files to determine the project structure and dependencies.
2. **Configuration**: Gradle configures the build by resolving dependencies, setting up tasks, and defining the project's properties and extensions.
3. **Execution**: Gradle executes the tasks defined in the build script in the order specified.
4. **Gradle Cache**: Gradle caches build artifacts and dependencies to speed up future builds.
5. **Testing**: Gradle runs any tests defined in the build script to ensure the code is functioning as expected.
6. **Build Output**: Gradle produces the build output, such as JAR, WAR or APK files, in the designated location.
7. **Cleanup**: Gradle performs cleanup tasks, such as removing temporary files and closing connections.

Overall, Gradle provides a flexible and efficient build system for managing complex software projects.

## Advantages of automated build process

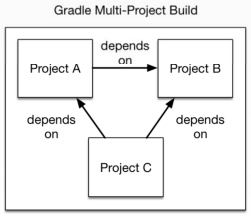
1. **Consistency**: Automated builds ensure that every build is performed the same way, using the same configuration and tools, and generating the same output. This reduces the risk of errors and ensures consistency across different environments.
2. **Efficiency**: Automated builds save time and effort by eliminating the need for manual interventions, such as copying files, running tests, and generating documentation. This allows developers to focus on more important tasks, such as writing code and fixing bugs.
3. **Speed**: Automated builds can be performed much faster than manual builds, especially for large and complex projects. This is because automated builds can be parallelized, optimized, and cached, reducing the build time significantly.
4. **Quality**: Automated builds can help improve the quality of the software by running tests, analyzing code, and detecting errors early in the development cycle. This can reduce the cost and time required to fix bugs later in the process.
5. **Scalability**: Automated builds can be easily scaled to support multiple branches, versions, and environments. This allows developers to quickly build and deploy software across different platforms, without compromising quality or consistency.

Overall, an automated build process can help improve the productivity, quality, and agility of software development teams, while reducing costs and risks.

## Manage Dependencies

## What is a build dependency?

Build dependencies are the software packages or libraries that are required to build a software application from its source code. These dependencies include compilers, build tools, and other software libraries that are needed to compile, link, and package the software application.






innovate achieve lead